

Patrones y Arquetipos – Cordillera Platform

Documento tecnico correspondiente al Parcial 2 de **Desarrollo Full Stack III (DSY1106)**.

1. Objetivo del documento

Este documento describe y justifica los patrones de diseno, patrones arquitectonicos y arquetipos utilizados en **Cordillera Platform**, plataforma desarrollada para Grupo Cordillera.

La solucion implementa una arquitectura basada en microservicios, con frontend ejecutivo, BFF Gateway, servicios backend especializados, persistencia independiente y despliegue mediante Docker Compose.

2. Contexto de la solucion

Grupo Cordillera posee informacion distribuida en distintos sistemas:

- POS.
- E-commerce.
- Inventario.
- Finanzas.
- CRM.

Cordillera Platform permite consolidar datos, calcular KPIs y generar reportes ejecutivos.

Flujo principal:

Usuario

```
-> Frontend React 19 + Nginx :3000
-> BFF Gateway :8081
-> Data Service :8083
-> KPI Service :8084
-> Report Service :8085
-> MySQL Docker :3307
```

3. Arquitectura general

La arquitectura se compone de:

Componente	Tecnologia	Puerto	Responsabilidad
Frontend	React 19 + Vite + Nginx	3000	Interfaz ejecutiva
BFF Gateway	Java 21 + Spring Boot + Maven	8081	Punto unico de entrada para frontend
Data Service	Java 21 + Spring Boot + Maven	8083	Gestion de datos operacionales
KPI Service	Java 21 + Spring Boot + Maven	8084	Gestion y calculo de KPIs
Report Service	Java 21 + Spring Boot + Maven	8085	Gestion y exportacion de reportes
MySQL	MySQL 8.4 Docker	3307	Persistencia de datos

4. API Gateway / BFF

Aplicacion en el proyecto

El patron **API Gateway / Backend For Frontend** se implementa en el servicio:

```
bff-gateway
```

Clases principales:

```
DashboardController
DashboardService
DashboardResponse
ReportesProxyController
ConfiguracionController
CorsConfig
```

Endpoints principales:

```
GET /api/dashboard/stats
GET /api/dashboard/kpis
GET /api/dashboard/sucursal/{id}
GET /api/reportes
GET /api/configuracion
```

Justificacion tecnica

El BFF Gateway permite que el frontend consuma un unico punto de entrada, evitando llamadas directas a cada microservicio.

Ventajas:

- Reduce acoplamiento entre frontend y microservicios.
- Centraliza integracion de datos.
- Facilita control de CORS.
- Simplifica el consumo HTTP desde React.
- Permite respuestas consolidadas para el dashboard ejecutivo.
- Mejora mantenibilidad porque los cambios internos de microservicios no obligan a modificar todo el frontend.

Aplicacion real validada

El frontend consume:

```
http://localhost:8081/api/dashboard/stats
```

El BFF consolida informacion desde:

```
Data Service -> http://data-service:8083  
KPI Service -> http://kpi-service:8084  
Report Service -> http://report-service:8085
```

5. Repository Pattern

Aplicacion en el proyecto

El patron **Repository Pattern** se aplica en los microservicios con Spring Data JPA.

Repositorios principales:

```
DatoRepository  
KpiRepository  
ReporteRepository
```

Ejemplo:

```
public interface ReporteRepository extends JpaRepository<Reporte, Long> {  
    List<Reporte> findByArea(String area);  
}
```

Justificacion tecnica

Repository Pattern separa la logica de persistencia de la logica de negocio.

Ventajas:

- Evita consultas directas dentro de los controladores.
- Mantiene el codigo mas limpio y ordenado.
- Facilita pruebas unitarias mediante mocks.
- Permite cambiar detalles de persistencia sin afectar servicios o controladores.
- Cumple con la separacion de responsabilidades.

Aplicacion por servicio

Servicio	Repository	Responsabilidad
data-service	DatoRepository	Persistir datos operacionales
kpi-service	KpiRepository	Persistir KPIs
report-service	ReporteRepository	Persistir reportes ejecutivos

6. Factory Method en KPI Service

Aplicacion en el proyecto

En `kpi-service`, Factory Method se usa para seleccionar calculadores de KPIs segun la categoria.

Clases principales:

```
KpiFactory  
KpiCalculator  
VentasCalculator  
InventarioCalculator  
LogisticaCalculator  
RentabilidadCalculator
```

Funcionamiento

`KpiFactory` centraliza la creacion o seleccion del calculador correspondiente.

Ejemplo conceptual:

```
categoria = "ventas" -> VentasCalculator
categoria = "inventario" -> InventarioCalculator
categoria = "logistica" -> LogisticaCalculator
categoria = "rentabilidad" -> RentabilidadCalculator
```

Justificacion tecnica

Ventajas:

- Evita condicionales repetidos en la logica principal.
- Facilita agregar nuevos tipos de KPIs.
- Aplica el principio abierto/cerrado.
- Mejora mantenibilidad del servicio.
- Permite encapsular calculos por categoria.

7. Factory Method en Report Service

Aplicacion en el proyecto

En `report-service` , Factory Method se usa para exportar reportes en distintos formatos.

Clases principales:

```
ExportadorFactory
Exportador
PdfExportador
ExcelExportador
JsonExportador
```

Funcionamiento

`ExportadorFactory` selecciona el exportador adecuado segun el formato solicitado.

Ejemplo conceptual:

```
formato = "PDF" -> PdfExportador
formato = "EXCEL" -> ExcelExportador
formato = "JSON" -> JsonExportador
```

Justificacion tecnica

Ventajas:

- Permite agregar nuevos formatos sin modificar el flujo principal.
- Evita acoplar `ReporteService` a clases concretas.
- Mejora extensibilidad.
- Facilita pruebas unitarias por exportador.
- Ordena la logica de exportacion.

Evidencia funcional

Se valido exportacion de reporte desde frontend, generando:

```
reporte-1.pdf
```

Contenido validado:

```
Reporte Ejecutivo Cordillera Platform
ID: 1
Titulo: Reporte Ejecutivo Mayo 2026
Tipo: PDF
Area: Gerencia
Valor: 1250000.00
Fecha Generacion: 2026-05-26T15:15:51.196017
```

8. Circuit Breaker con Resilience4j

Aplicacion en el proyecto

Se implementa Circuit Breaker en integraciones internas entre microservicios.

Relaciones principales:

```
kpi-service -> data-service  
report-service -> kpi-service
```

Tecnologia utilizada:

```
Resilience4j
```

Estados del Circuit Breaker

```
CLOSED -> OPEN -> HALF-OPEN
```

Justificacion tecnica

El Circuit Breaker evita que una falla en un microservicio genere una caida en cascada.

Ventajas:

- Mejora resiliencia.
- Permite respuestas degradadas.
- Evita saturar servicios fallando.
- Aumenta estabilidad de la plataforma.
- Facilita defensa tecnica de disponibilidad.

Aplicacion en defensa oral

Se puede explicar así:

Si KPI Service no logra obtener datos desde Data Service, el Circuit Breaker evita que el error se propague. En vez de romper toda la plataforma, se activa un fallback y el sistema puede seguir respondiendo de forma controlada.

9. Arquitectura por capas

Aplicacion en backend

Cada microservicio sigue la separacion:

```
Controller -> Service -> Repository -> Model
```

Responsabilidad por capa

Capa	Responsabilidad
Controller	Exponer endpoints REST
Service	Implementar logica de negocio
Repository	Acceder a base de datos
Model	Representar entidades JPA
DTO	Transportar datos entre capas o servicios
Config	Centralizar configuracion tecnica

Justificacion tecnica

Ventajas:

- Codigo mas ordenado.
- Mayor mantenibilidad.
- Facilita pruebas unitarias.
- Evita mezclar responsabilidades.
- Permite que cada integrante trabaje en un servicio sin afectar otros componentes.

10. Database-per-service

Aplicacion en el proyecto

Cada microservicio con persistencia tiene su propia base de datos.

Servicio	Base de datos
data-service	data_db
kpi-service	kpi_db
report-service	report_db

Justificacion tecnica

Ventajas:

- Independencia de datos.

- Menor acoplamiento.
- Escalabilidad por servicio.
- Mantenimiento mas simple.
- Evita acceso directo a tablas internas de otro servicio.

11. Arquetipo Maven base

Origen del arquetipo

El arquetipo Maven usado se basa en el procedimiento entregado por el profesor.

Procedimiento principal:

```
mvn archetype:create-from-project
```

Luego se instala desde:

```
target/generated-sources/archetype
```

Con:

```
mvn clean install
```

Y permite crear nuevos microservicios mediante:

```
mvn --% archetype:generate -DarchetypeCatalog=local -DarchetypeGroupId=cl
```

Aplicacion en Cordillera Platform

Servicios alineados al arquetipo:

```
data-service  
kpi-service  
report-service  
bff-gateway
```

Estructura comun:

```
src/main/java
src/main/resources
src/test/java
pom.xml
Dockerfile
.dockerignore
```

Paquetes comunes:

```
controller
service
repository
model
dto
config
```

Justificacion tecnica

Ventajas:

- Estandariza los microservicios.
- Facilita crear nuevos componentes.
- Reduce errores de estructura.
- Permite reutilizar buenas practicas.
- Mejora coherencia del proyecto para la entrega.

12. Frontend basado en componentes

Aplicacion en el proyecto

El frontend esta desarrollado con:

```
React 19 + Vite + NPM
```

Se sirve en Docker mediante:

```
Nginx
```

Puerto:

3000

Componentes reutilizables

Ejemplos:

```
ExecutiveDashboard  
TrendPanel  
MetricCard  
StatusBadge  
ReportItem  
ServiceStatusCard  
AlertItem  
SectionHeader
```

Servicios API

El consumo HTTP esta centralizado en:

```
frontend/src/services/dashboardApi.js
```

El frontend consume unicamente el BFF Gateway:

```
VITE_API_BASE_URL=http://localhost:8081
```

Justificacion tecnica

Ventajas:

- Reutilizacion de componentes.
- Separacion entre UI y consumo HTTP.
- Mejor mantenibilidad.
- Mayor claridad visual.
- Permite evolucion de pantallas sin romper integracion.
- Facilita demostrar dashboard, KPIs, reportes, alertas y servicios.

13. Docker Compose

Aplicacion en el proyecto

La arquitectura completa se levanta con:

```
docker compose up -d --build
```

Servicios definidos:

```
mysql
data-service
kpi-service
report-service
bff-gateway
frontend
```

Puertos finales

Componente	Puerto
Frontend React 19 + Nginx	3000
BFF Gateway	8081
Data Service	8083
KPI Service	8084
Report Service	8085
MySQL host	3307

Justificacion tecnica

Docker Compose permite ejecutar la solucion completa sin depender de XAMPP ni configuraciones locales complejas.

Ventajas:

- Instalacion mas simple.
- Misma ejecucion en distintos PCs.
- Evita conflictos con MySQL local.
- Permite levantar toda la arquitectura con un comando.
- Facilita presentacion ante el docente.

14. Evidencia de validacion

Se valido la arquitectura con:

```
docker compose up -d --build
docker compose ps
```

Servicios operativos:

```
frontend      0.0.0.0:3000->3000/tcp
bff-gateway   0.0.0.0:8081->8081/tcp
data-service  0.0.0.0:8083->8083/tcp
kpi-service   0.0.0.0:8084->8084/tcp
report-service 0.0.0.0:8085->8085/tcp
mysql         0.0.0.0:3307->3306/tcp
```

Endpoints validados:

```
GET http://localhost:8083/api/datos
GET http://localhost:8084/api/kpis
GET http://localhost:8085/api/reportes
GET http://localhost:8081/api/dashboard/stats
GET http://localhost:8081/api/reportes
GET http://localhost:8081/api/configuracion
```

Resultados:

- Data Service responde con datos operacionales.
- KPI Service responde con KPIs.
- Report Service responde con reportes.
- BFF Gateway consolida dashboard.
- Frontend muestra KPIs, servicios, alertas y reportes.
- Descarga PDF funcionando.
- Docker Compose levanta toda la arquitectura.

15. Relacion con la rubrica

Indicador 1

Se implementan y justifican al menos tres patrones:

- API Gateway / BFF.
- Repository Pattern.
- Factory Method.
- Circuit Breaker.
- Arquitectura por capas.

Indicador 2

La solución usa arquetipos Maven coherentes:

- Un BFF.
- Microservicios Maven independientes.
- Separación por capas.
- Docker por componente.
- Database-per-service.

Indicador 4

Buenas prácticas:

- Código separado por capas.
- Pruebas unitarias.
- JaCoCo.
- Docker Compose.
- README por componente.
- Endpoints validados.

Defensa oral

Este documento apoya:

- Explicación de patrones.
- Justificación de arquitectura.
- Escalabilidad.
- Mantenibilidad.
- Resiliencia.
- Evidencia técnica de ejecución.

16. Conclusion

Cordillera Platform aplica una arquitectura de microservicios coherente con los requerimientos del Parcial 2.

Los patrones implementados permiten mejorar:

- Mantenibilidad.
- Escalabilidad.
- Resiliencia.
- Separacion de responsabilidades.
- Claridad tecnica para defensa oral.

La solucion queda validada con Docker Compose, frontend operativo, BFF funcional, microservicios conectados, reportes exportables y documentacion tecnica alineada con la rubrica.